CIS*3110 Operating Systems

Week 6

Lecture #11 Virtualization: Page faults, swapping

Prof. Yulia Kotseruba

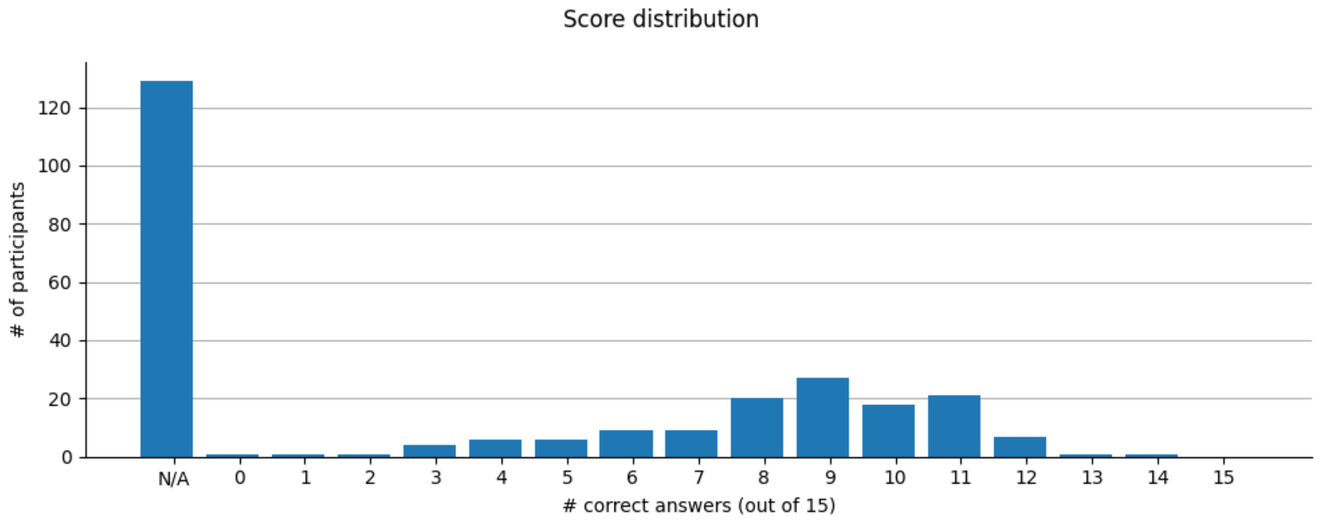
February 12, 2026

Quiz 9

- **56%** average (down from 57% in quiz 8)
- 3 hardest questions:
  - (29%) What is the guaranteed complexity of selecting the next process in CFS? a) O(1) b) O(n) c) O(log n) d) O(n^2)
  - (39%) How many times will "X" be printed? a) 1 b) 2 c) 3 d) 4
  - (41%) How much memory is indirectly lost when this code executes? a) 10 b) 30 c) 40 d) none

```c
int main() {
    if (fork() || fork()) {
        printf("X\n");
    }
}
```

```c
int main(){
    int **arr = malloc(40);
    arr[0] = malloc(10);
    arr = malloc(30);
    free(arr);
    return 0;
}
```



Recap

- Virtual memory is **larger** than physical memory
- We still need to store information for virtual pages somewhere
- Swapping - since RAM is too small, use the **hard drive** to extend it
- RAM becomes a **cache** for virtual memory
  - **Page faults** occur because the requested page is not in RAM
  - page needs to be brought to RAM from the hard drive
  - if RAM is full, need to remove pages to make room

Recap

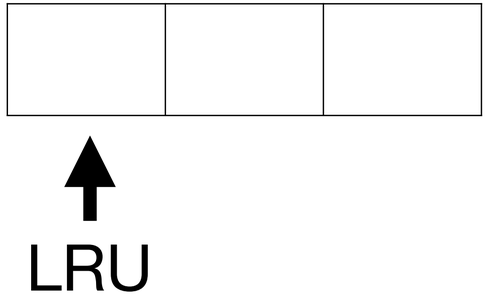
- Replacement policies
  - **MIN** - remove pages that will not be used in the future - infeasible in real world
  - **FIFO** - remove oldest pages - simple but **ineffective** for most workloads
  - **RAND** - remove **random** pages - simple, but **inconsistent** by design
  - **LRU** - remove **least recently** used pages - use **history** to make eviction decisions

LRU

- **Least recently used** - use history to guide selection
  - if a page was accessed **recently**, it is likely to be accessed **again**
  - evict the least recently used page
  - relies on **temporal locality**
    - e.g. if a process loops over the same structure frequently

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1



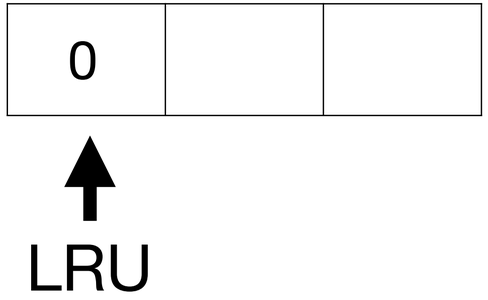
0 1 2 0 1 3 0 3 1 2 1

cache sorted by recency

Access	Hit/ Miss?	Evict	Cache
0			

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

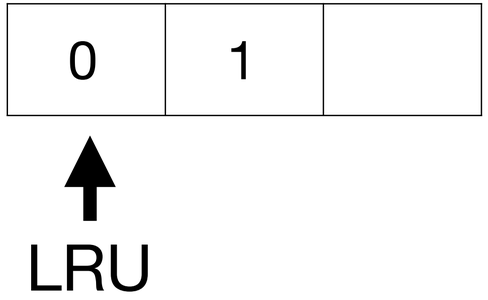


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

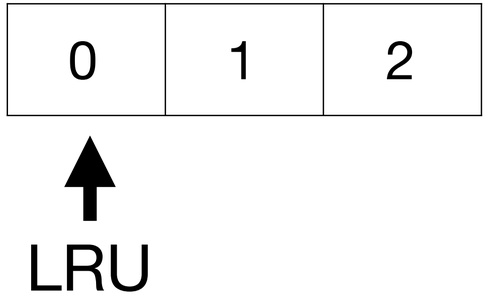


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

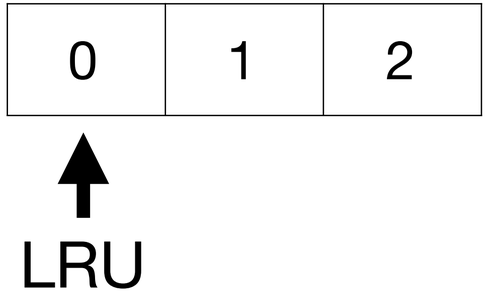


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1



0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

1	2	0

LRU

0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

1	2	0

LRU

0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1			

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

1	2	0

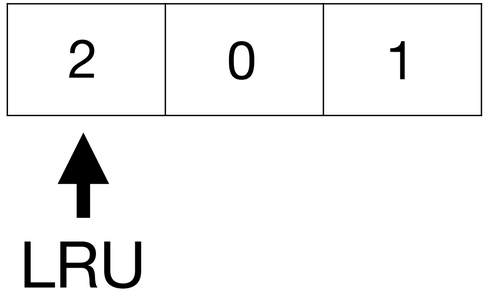
LRU

0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

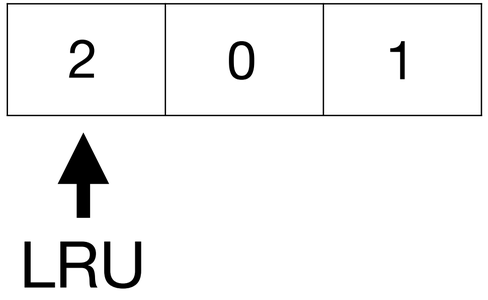


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

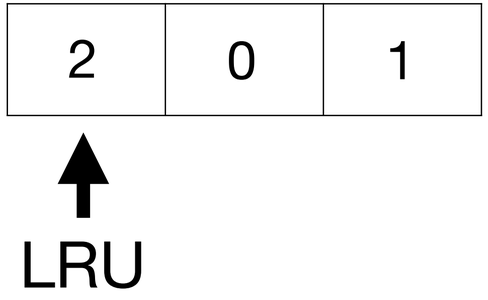


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3			

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

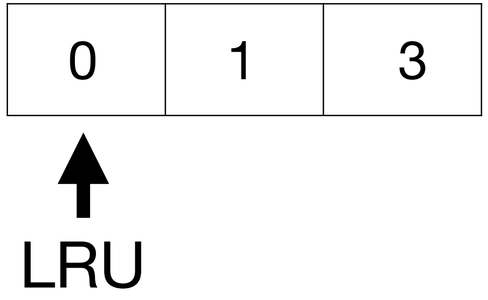


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss		

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

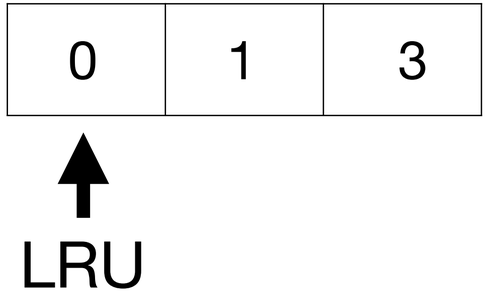


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

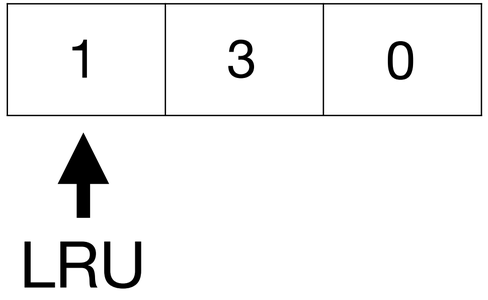


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0			

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

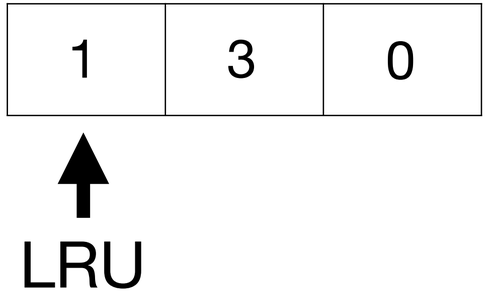


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

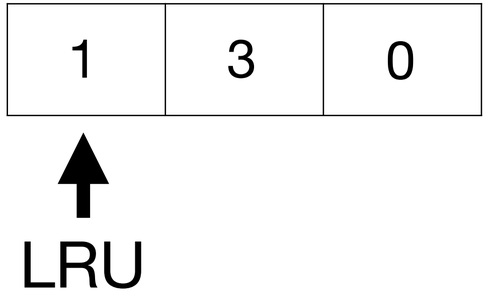


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3			

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

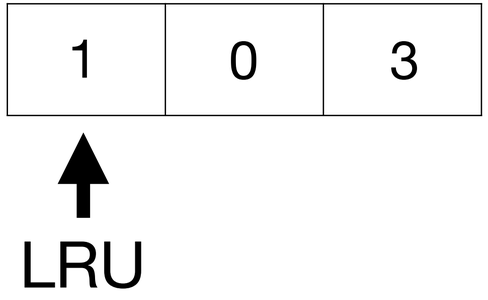


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

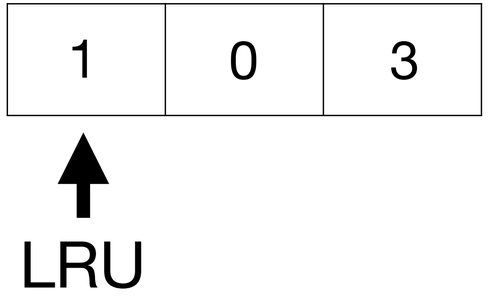


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

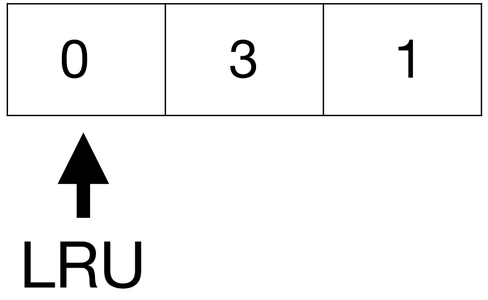


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3
1	Hit		

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

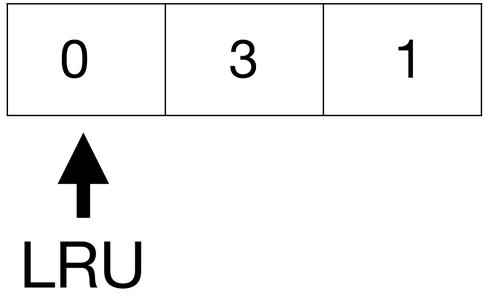


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3
1	Hit		0,3,1

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

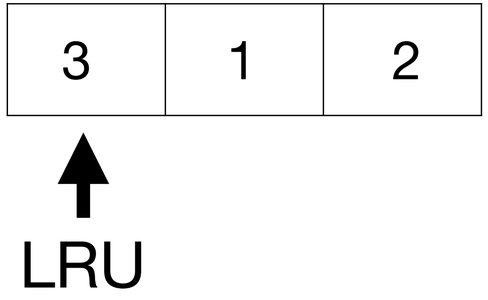


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3
1	Hit		0,3,1
2	Miss		

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

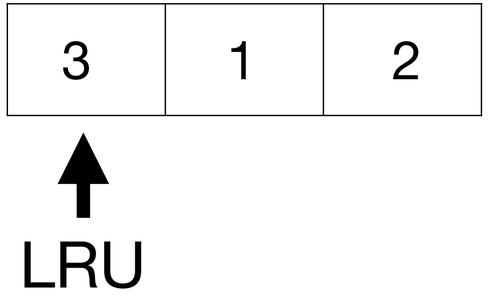


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3
1	Hit		0,3,1
2	Miss	0	3,1,2

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

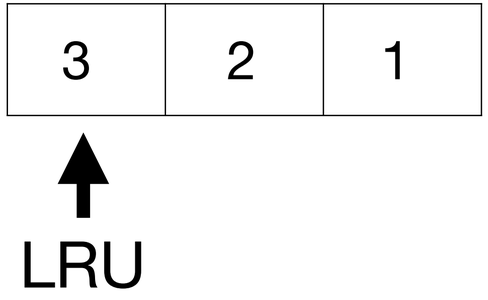


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3
1	Hit		0,3,1
2	Miss	0	3,1,2
1	Hit		

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

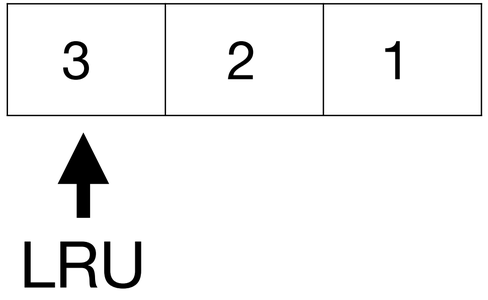


0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3
1	Hit		0,3,1
2	Miss	0	3,1,2
1	Hit		3,2,1

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1



0 1 2 0 1 3 0 3 1 2 1

after every hit, move the page to the end of the list

after miss, replace the page and move the new page to the end of the list

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3
1	Hit		0,3,1
2	Miss	0	3,1,2
1	Hit		3,2,1

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

What is the hit rate?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}}$$

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3
1	Hit		0,3,1
2	Miss	0	3,1,2
1	Hit		3,2,1

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

What is the hit rate?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}} = \frac{6}{6+5} = 54.5\,\%$$

What is the hit rate without the compulsory misses?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Noncompulsory Misses}} =$$

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3
1	Hit		0,3,1
2	Miss	0	3,1,2
1	Hit		3,2,1

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

The first access to any page is compulsory

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3
1	Hit		0,3,1
2	Miss	0	3,1,2
1	Hit		3,2,1

What is the hit rate?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}} = \frac{6}{6+5} = 54.5\,\%$$

What is the hit rate without the compulsory misses?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Noncompulsory Misses}} =$$

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

The first access to any page is compulsory

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3
1	Hit		0,3,1
2	Miss	0	3,1,2
1	Hit		3,2,1

What is the hit rate?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}} = \frac{6}{6+5} = 54.5\,\%$$

What is the hit rate without the compulsory misses?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Noncompulsory Misses}} = \frac{6}{6+1} = 85.7\,\%$$

LRU

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3
1	Hit		0,3,1
2	Miss	0	3,1,2
1	Hit		3,2,1

What is the hit rate?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}} = \frac{6}{6+5} = 54.5\,\%$$

This is the same as MIN

What is the hit rate without the compulsory misses?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Noncompulsory Misses}} = \frac{6}{6+1} = 85.7\,\%$$

this sequence is artificially constructed to get the exact same result, but LRU approximates MIN well in general

LRU

- **Least recently used** - use history to guide selection
  - if a page was accessed **recently**, it is likely to be accessed **again**
  - evict the least recently used page
  - relies on **temporal locality**
    - e.g. if a process loops over the same structure frequently

LRU

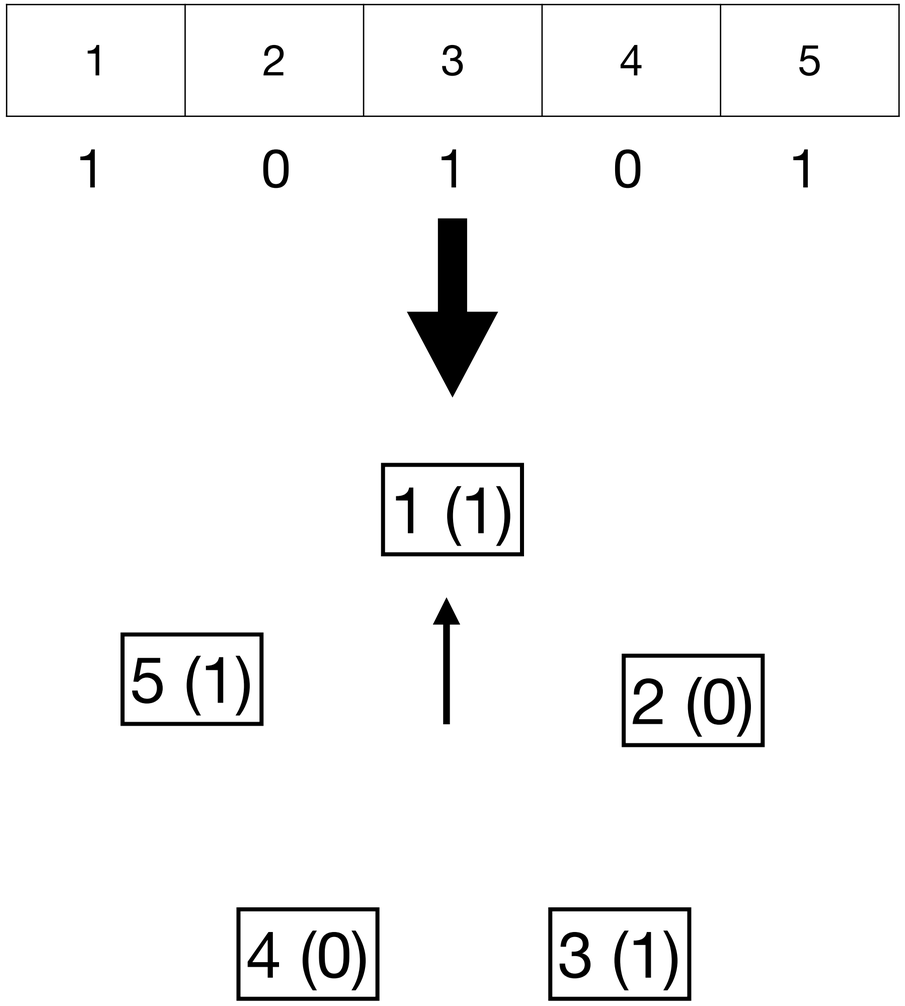
- How to implement?
  - Option #1: record when the page was accessed, search for the least recently used when need to evict
  - Option #2: rearrange the cache such that the least recently used page is always at the top
- Both options are very **expensive**
  - if RAM is 4GB with 4KB pages -> **1M pages to sort**

Approximating LRU

- Instead of sorting/searching the page table
- Use the "**referenced bit**" to estimate how recently the page has been used
  - 0=not referenced, 1=referenced
- Keep processes that have been referenced

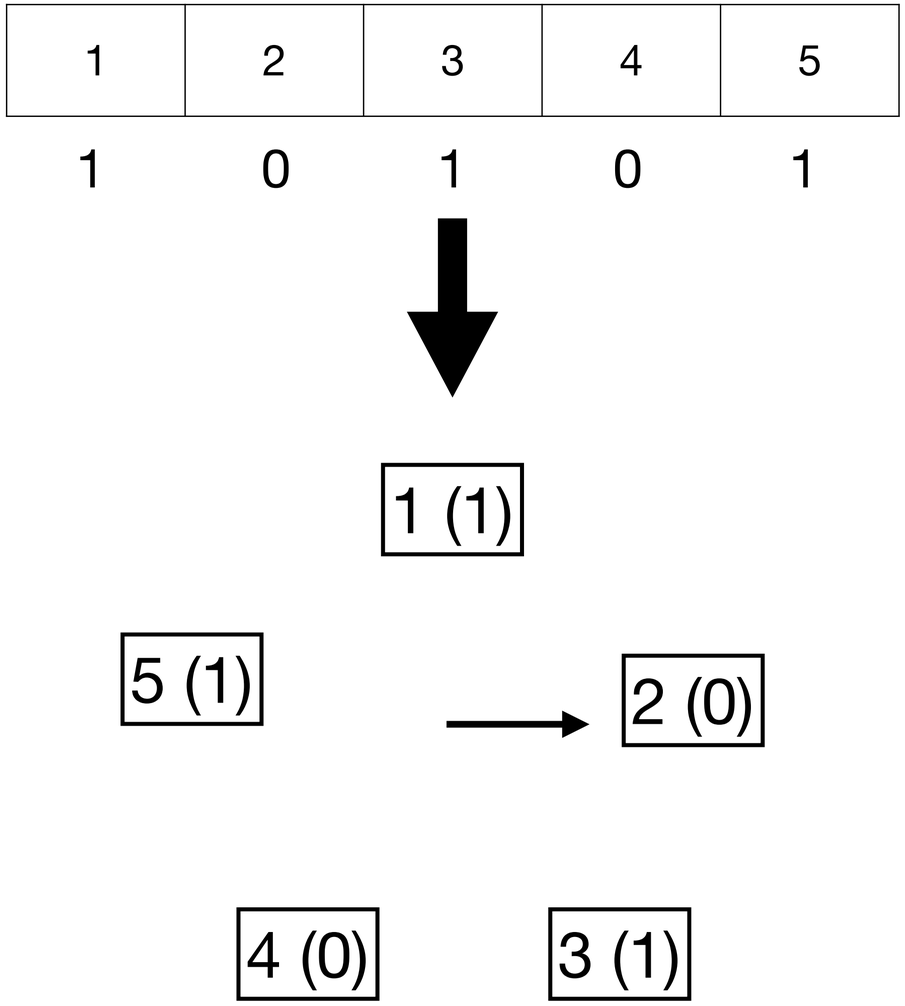
Clock algorithm

- Page table is treated as a **circular array**
- A pointer goes around the array (like a clock hand)
- When a page needs to be replaced
  - if the hand is pointing at a page with reference bit 1
    - set the bit to 0, move hand forward
  - if the hand is pointing at a page with reference bit 0
    - means it has not been referenced for some time
    - kick the page out and replace with new page
    - set the bit of the page to 0, move hand forward



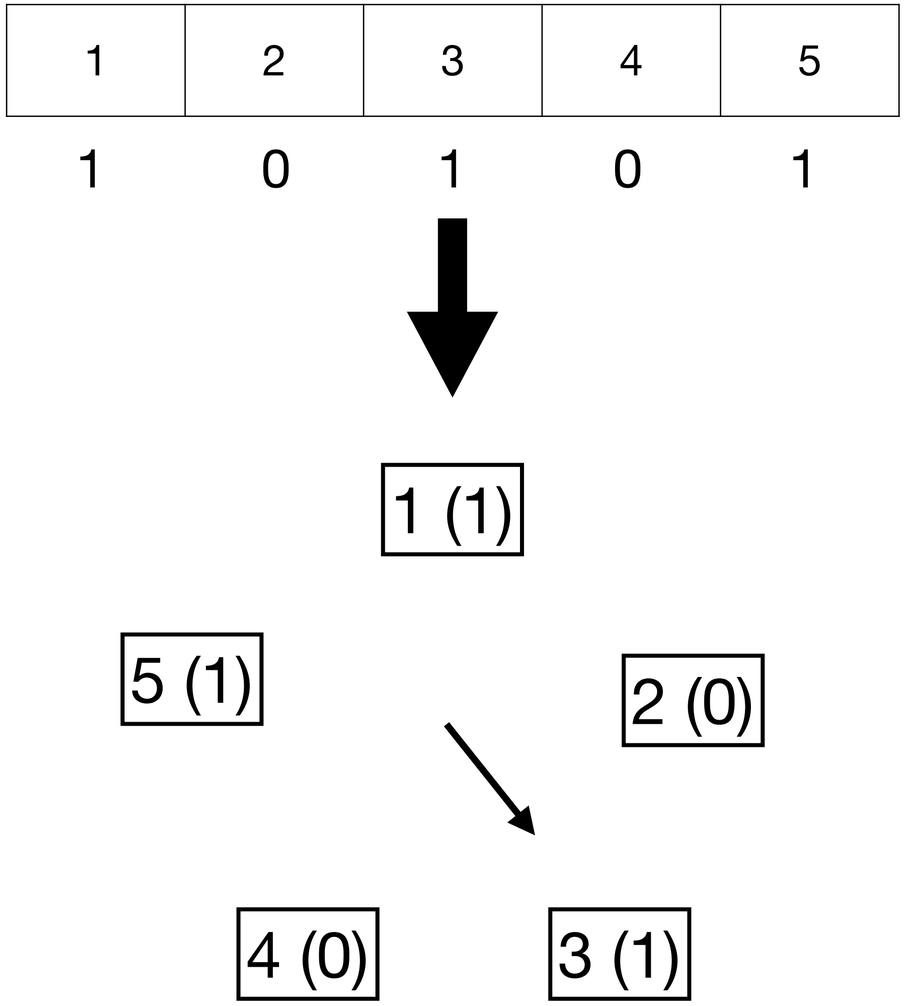
Clock algorithm

- Page table is treated as a **circular array**
- A pointer goes around the array (like a clock hand)
- When a page needs to be replaced
  - if the hand is pointing at a page with reference bit 1
    - set the bit to 0, move hand forward
  - if the hand is pointing at a page with reference bit 0
    - means it has not been referenced for some time
    - kick the page out and replace with new page
    - set the bit of the page to 0, move hand forward



Clock algorithm

- Page table is treated as a **circular array**
- A pointer goes around the array (like a clock hand)
- When a page needs to be replaced
  - if the hand is pointing at a page with reference bit 1
    - set the bit to 0, move hand forward
  - if the hand is pointing at a page with reference bit 0
    - means it has not been referenced for some time
    - kick the page out and replace with new page
    - set the bit of the page to 0, move hand forward

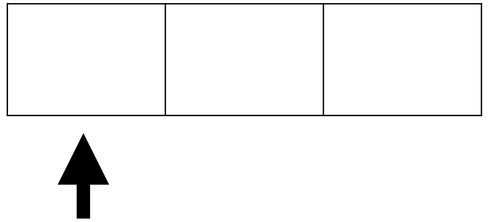


Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit

cache



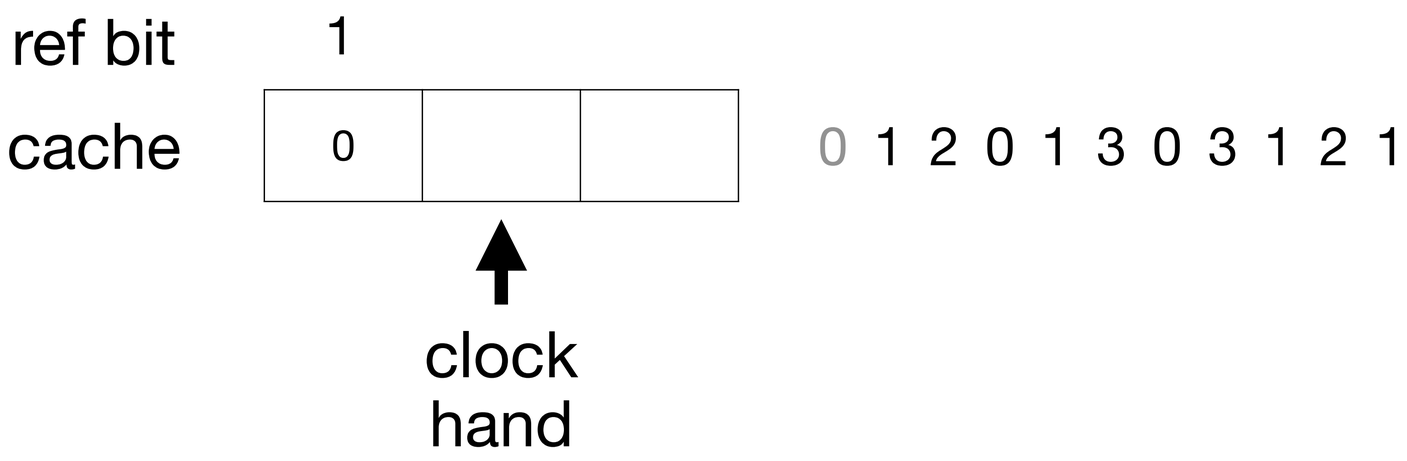
clock hand

0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0

Clock algorithm

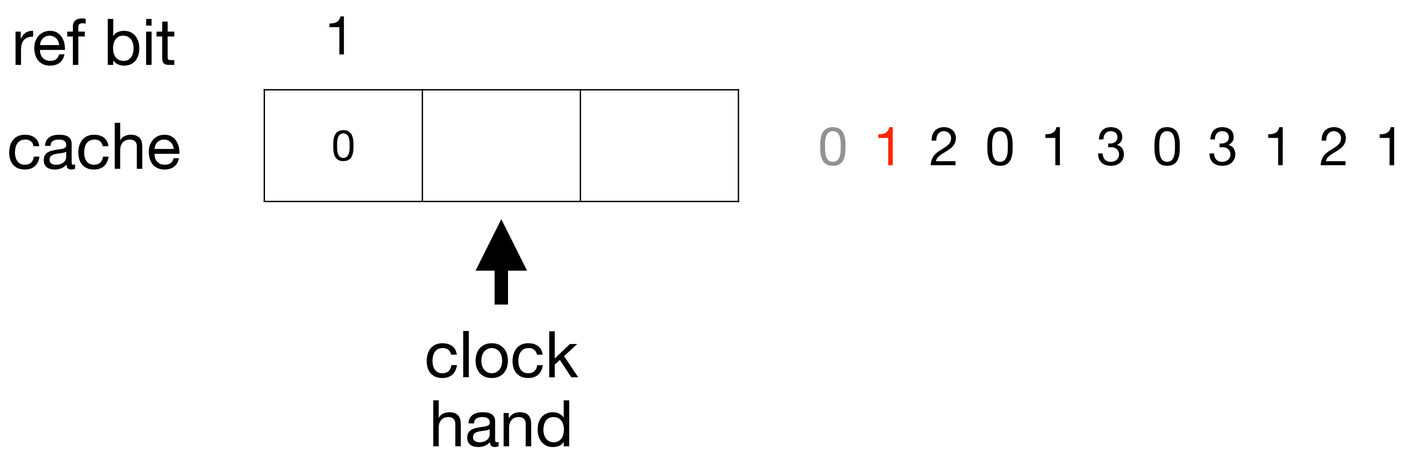
- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1



Access	Hit/ Miss?	Evict	Cache
0	Miss		0

Clock algorithm

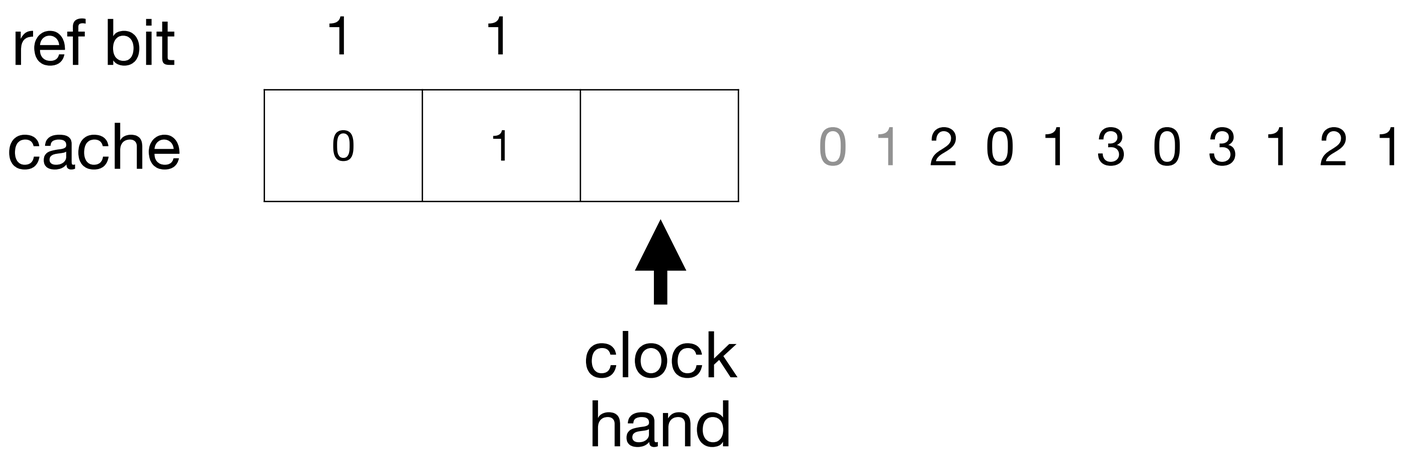
- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1



Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1			

Clock algorithm

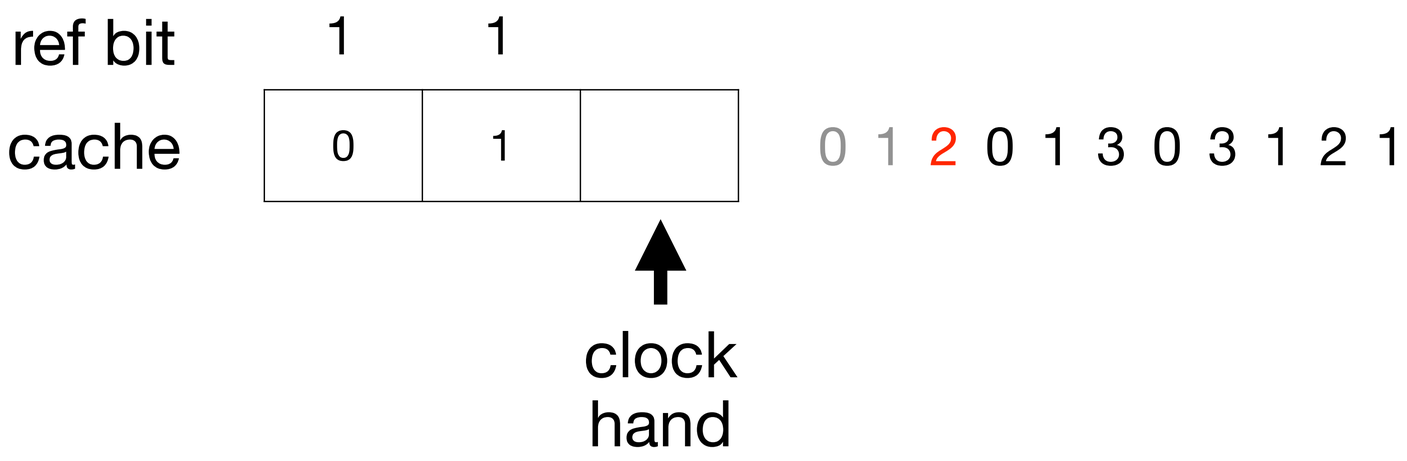
- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1



Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1

Clock algorithm

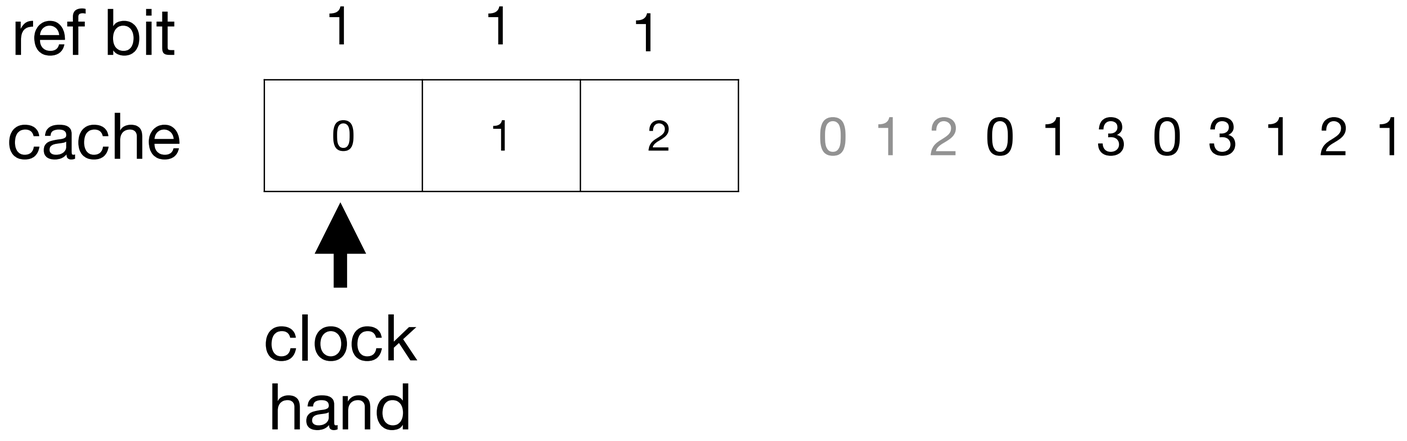
- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1



Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2			

Clock algorithm

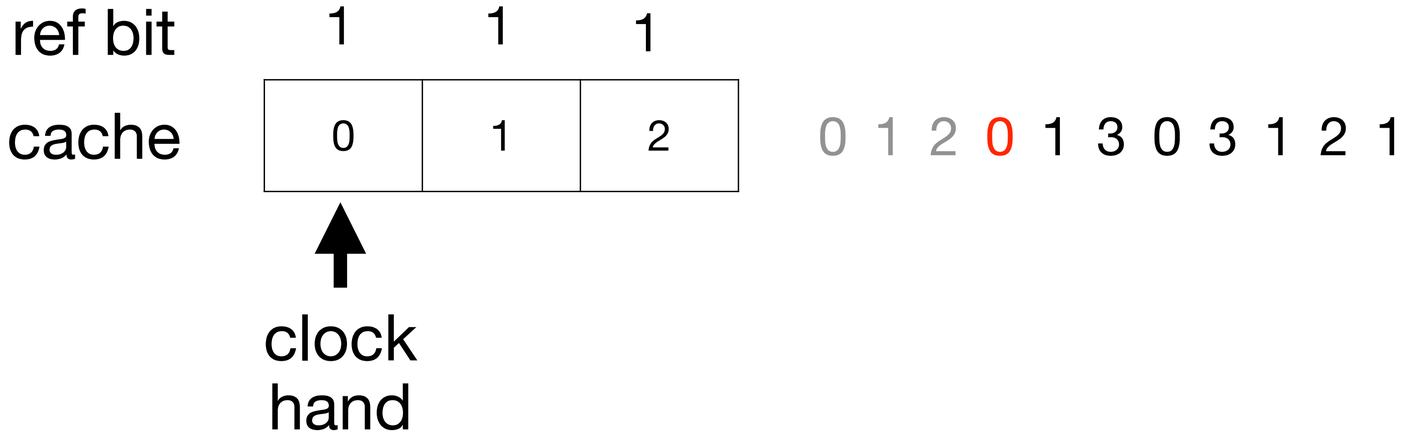
- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1



Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2

Clock algorithm

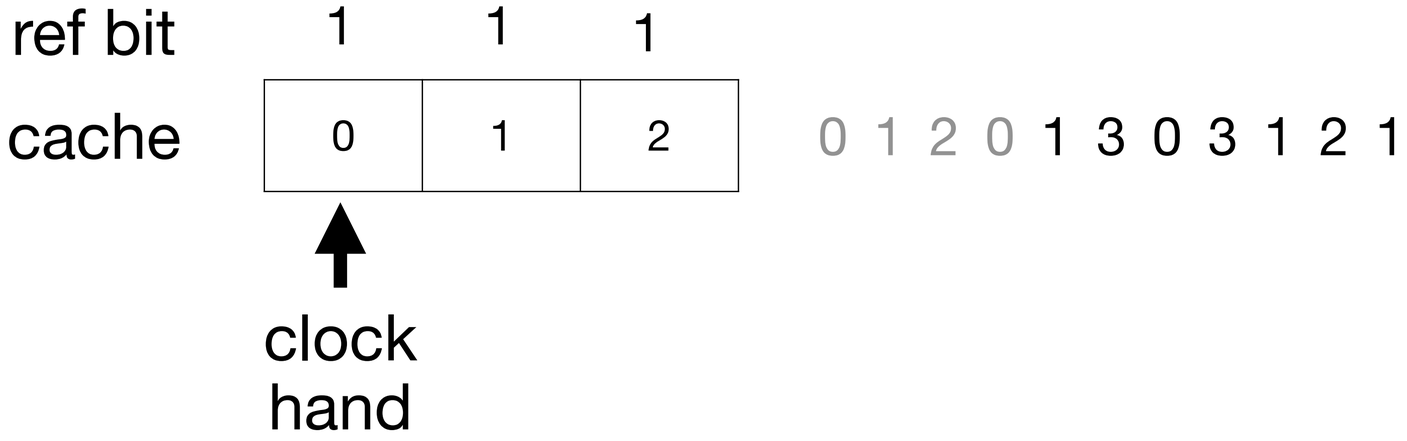
- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1



Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0			

Clock algorithm

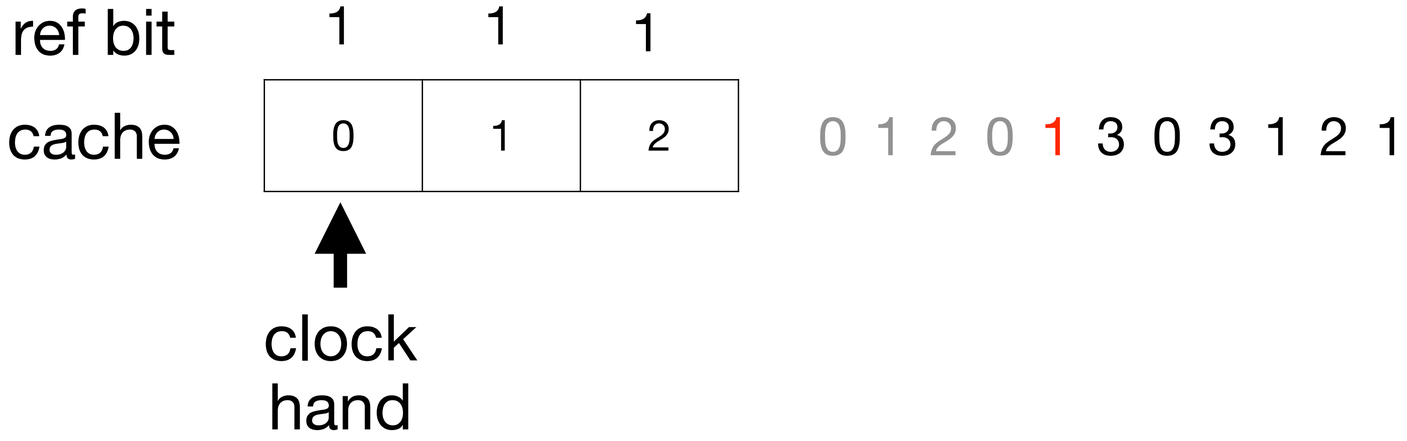
- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1



Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2

Clock algorithm

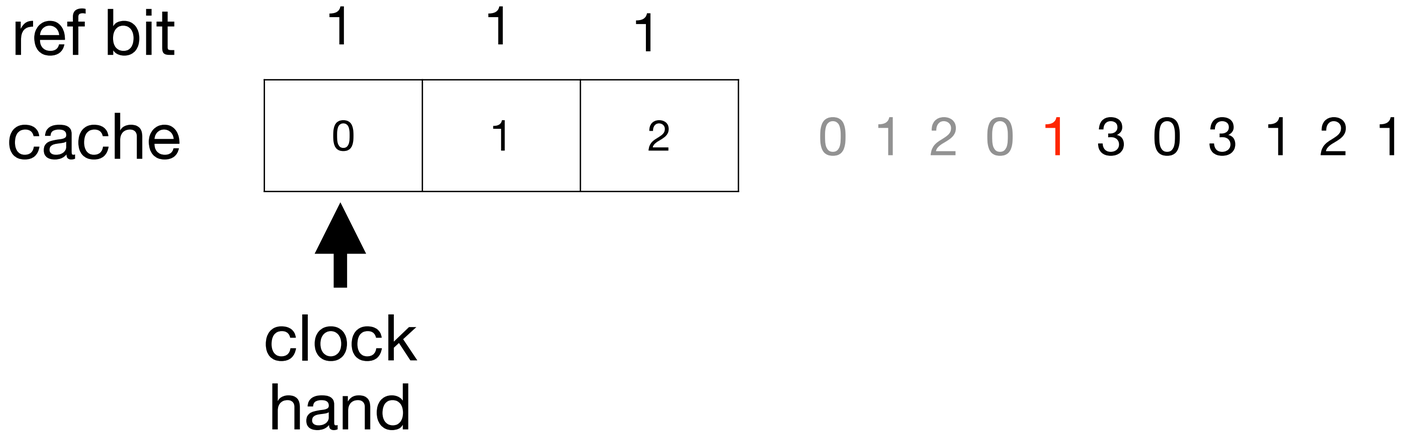
- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1



Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1			

Clock algorithm

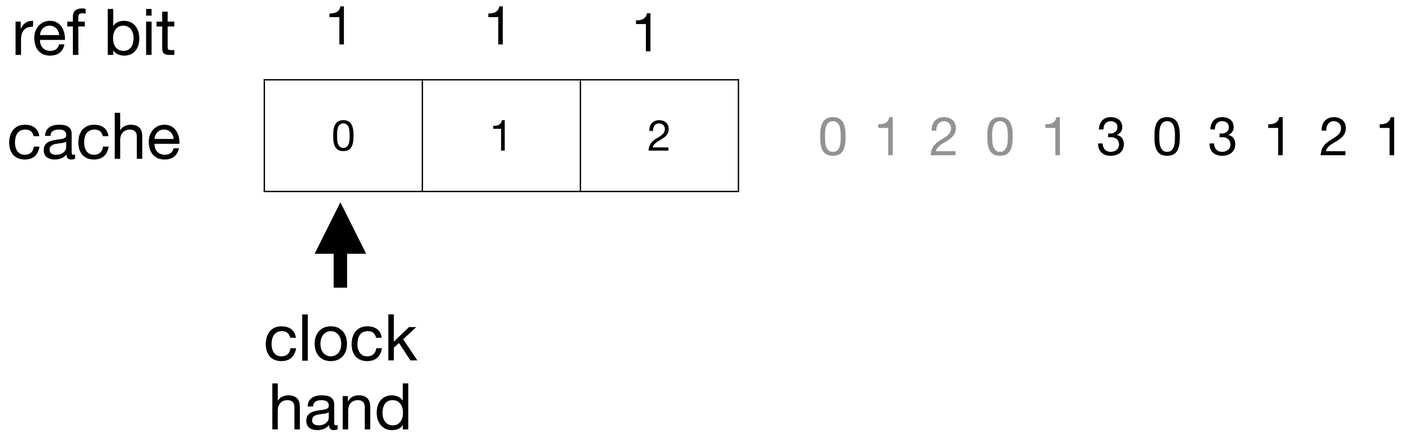
- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1



Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1



Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	1	1	1
cache	0	1	2

clock hand

0 1 2 0 1 3 0 3 1 2 1

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3			

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	0	1	1
cache	0	1	2

clock hand

0 1 2 0 1 3 0 3 1 2 1

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3			

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	0	0	1
cache	0	1	2

0 1 2 0 1 3 0 3 1 2 1

clock hand

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3			

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	0	0	0
cache	0	1	2

clock hand

0 1 2 0 1 3 0 3 1 2 1

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss		

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	1	0	0
cache	3	1	2

clock hand

0 1 2 0 1 3 0 3 1 2 1

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	1	0	0
cache	3	1	2

clock hand

0 1 2 0 1 3 0 3 1 2 1

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	1	0	0
cache	3	1	2

clock hand

0 1 2 0 1 3 0 3 1 2 1

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	1	0	0
cache	3	1	2

clock hand

0 1 2 0 1 3 0 3 1 2 1

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0			

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	1	1	0
cache	3	0	2

0 1 2 0 1 3 0 3 1 2 1

clock hand

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	1	1	0
cache	3	0	2

0 1 2 0 1 3 0 3 1 2 1

clock hand

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3			

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	1	1	0
cache	3	0	2

0 1 2 0 1 3 0 3 1 2 1

clock hand

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3	Hit		3,0,2

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	1	1	0
cache	3	0	2

0 1 2 0 1 3 0 3 1 2 1

clock hand

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3	Hit		3,0,2
1	Miss		

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	1	1	1
cache	3	0	2

0 1 2 0 1 3 0 3 1 2 1

clock hand

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3	Hit		3,0,2
1	Miss	2	3,0,1

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	1	1	1
cache	3	0	1

clock hand

0 1 2 0 1 3 0 3 1 2 1

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3	Hit		3,0,2
1	Miss	2	3,0,1

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	1	1	1
cache	3	0	1

clock hand

0 1 2 0 1 3 0 3 1 2 1

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3	Hit		3,0,2
1	Miss	2	3,0,1
2	Miss		

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	0	1	1
cache	3	0	1

clock hand

0 1 2 0 1 3 0 3 1 2 1

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3	Hit		3,0,2
1	Miss	2	3,0,1
2	Miss		

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	0	0	1
cache	3	0	1

clock hand

0 1 2 0 1 3 0 3 1 2 1

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3	Hit		3,0,2
1	Miss	2	3,0,1
2	Miss		

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	0	0	0
cache	3	0	1

0 1 2 0 1 3 0 3 1 2 1

clock hand

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3	Hit		3,0,2
1	Miss	2	3,0,1
2	Miss		

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	0	0	0
cache	3	0	1

clock hand

0 1 2 0 1 3 0 3 1 2 1

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3	Hit		3,0,2
1	Miss	2	3,0,1
2	Miss		

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	1	0	0
cache	2	0	1

clock hand

0 1 2 0 1 3 0 3 1 2 1

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3	Hit		3,0,2
1	Miss	2	3,0,1
2	Miss	3	2,0,1

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	1	0	0
cache	2	0	1

clock hand

0 1 2 0 1 3 0 3 1 2 1

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3	Hit		3,0,2
1	Miss	2	3,0,1
2	Miss	3	2,0,1
1	Hit		

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

ref bit	1	0	1
cache	2	0	1

clock hand

0 1 2 0 1 3 0 3 1 2 1

if ref bit = 1, reset it to 0, move hand

if ref bit = 0, it has not been accessed recently -> evict

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3	Hit		3,0,2
1	Miss	2	3,0,1
2	Miss	3	2,0,1
1	Hit		2,0,1

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

What is the hit rate?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}}$$

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3	Hit		3,0,2
1	Miss	2	3,0,1
2	Miss	3	2,0,1
1	Hit		2,0,1

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

What is the hit rate?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}} = \frac{4}{4+7} = 36.6\,\%$$

What is the hit rate without the compulsory misses?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Noncompulsory Misses}} =$$

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3	Hit		3,0,2
1	Miss	2	3,0,1
2	Miss	3	2,0,1
1	Hit		2,0,1

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

What is the hit rate?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}} = \frac{4}{4+7} = 36.6\,\%$$

What is the hit rate without the compulsory misses?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Noncompulsory Misses}} = \frac{4}{4+3} = 57.14\,\%$$

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3	Hit		3,0,2
1	Miss	2	3,0,1
2	Miss	3	2,0,1
1	Hit		2,0,1

Clock algorithm

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0,1
2	Miss		0, 1, 2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	3,1,2
0	Miss	1	3,0,2
3	Hit		3,0,2
1	Miss	2	3,0,1
2	Miss	3	2,0,1
1	Hit		2,0,1

What is the hit rate?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}} = \frac{4}{4+7} = 36.6\,\%$$

This is the same as FIFO :(

but usually it works quite well

What is the hit rate without the compulsory misses?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Noncompulsory Misses}} = \frac{4}{4+3} = 57.14\,\%$$

Clock algorithm

- A common **LRU approximation**
- Works well for most workloads
- Supported by most hardware (only requires 1 extra bit in a page table)

Clock algorithm variations

- **Two-handed** clock
  - the first hand is the same as before (but does not clear reference bits)
  - the second hand **clears reference bits**
  - the distance between them is kept **constant**
  - if set **too short** will degrade to **FIFO**, if **too long**, will be **inefficient**

Clock algorithm variations

- Clock with **dirty** bit
  - Hand operates as before
  - In addition to reference bit add a dirty bit
    - A page with dirty bit set to 1 has been modified and must be written to disk
    - This is expensive
  - When choosing a page to evict
    - Select first those that have both reference and dirty bit set to 0

Thrashing

- Thrashing is when a system is **constantly page faulting**
  - the process uses **all of its CPU** time for **swapping** pages
  - happens when **memory demands** of the process far **exceed RAM**
- Solution
  - buy **more RAM**
  - **kill** the trashing process (that’s what Linux does)

Benchmark

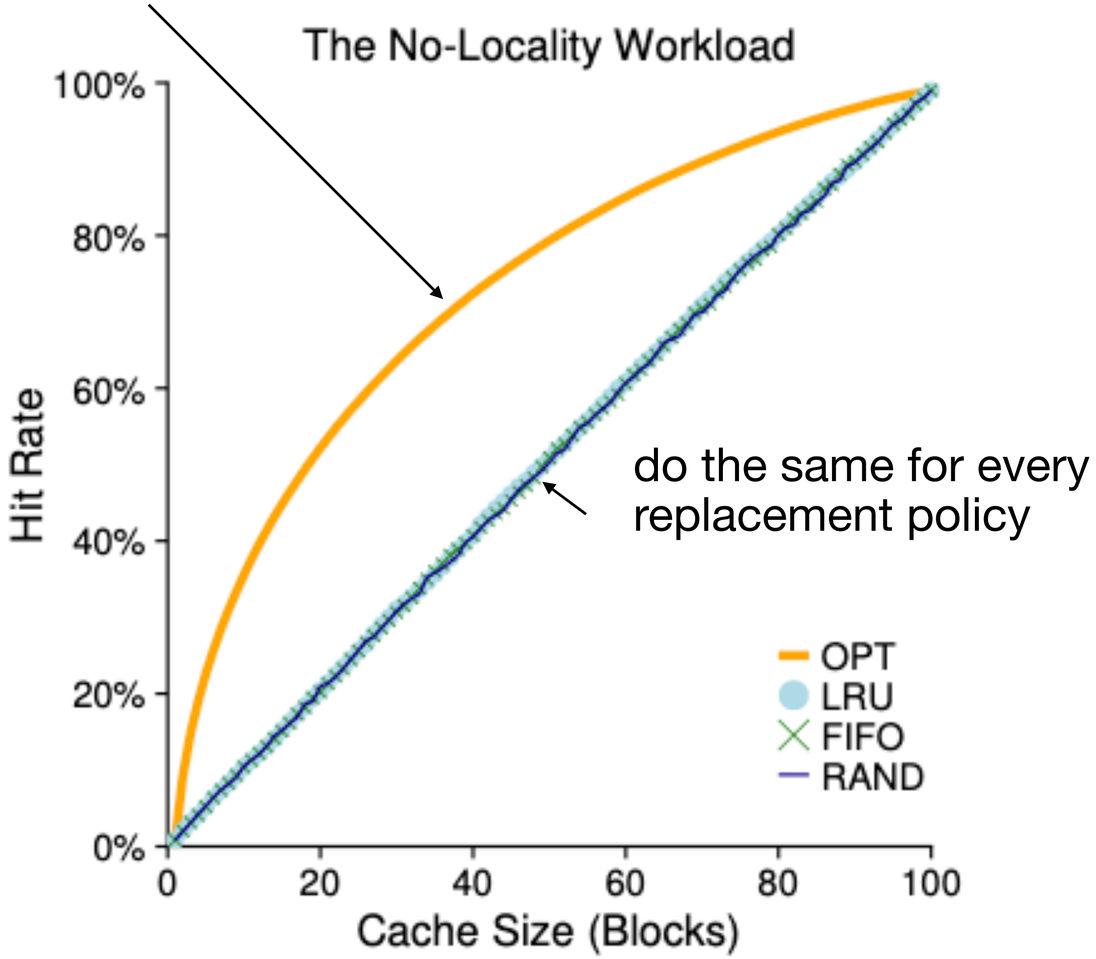
- Problem: It is **very difficult to predict** how these algorithms will behave on complex real-world workloads
- Solution: **benchmark**
  - design challenging tests
  - establish metric(s) for measuring performance
  - run many algorithms on the same tests and compute their metrics
  - use metric(s) to judge their performance
    - relative to one another
    - relative to some baseline

Benchmark

run program with 100 different caches sizes and MIN policy -> compute hit rate -> plot a dot for each run

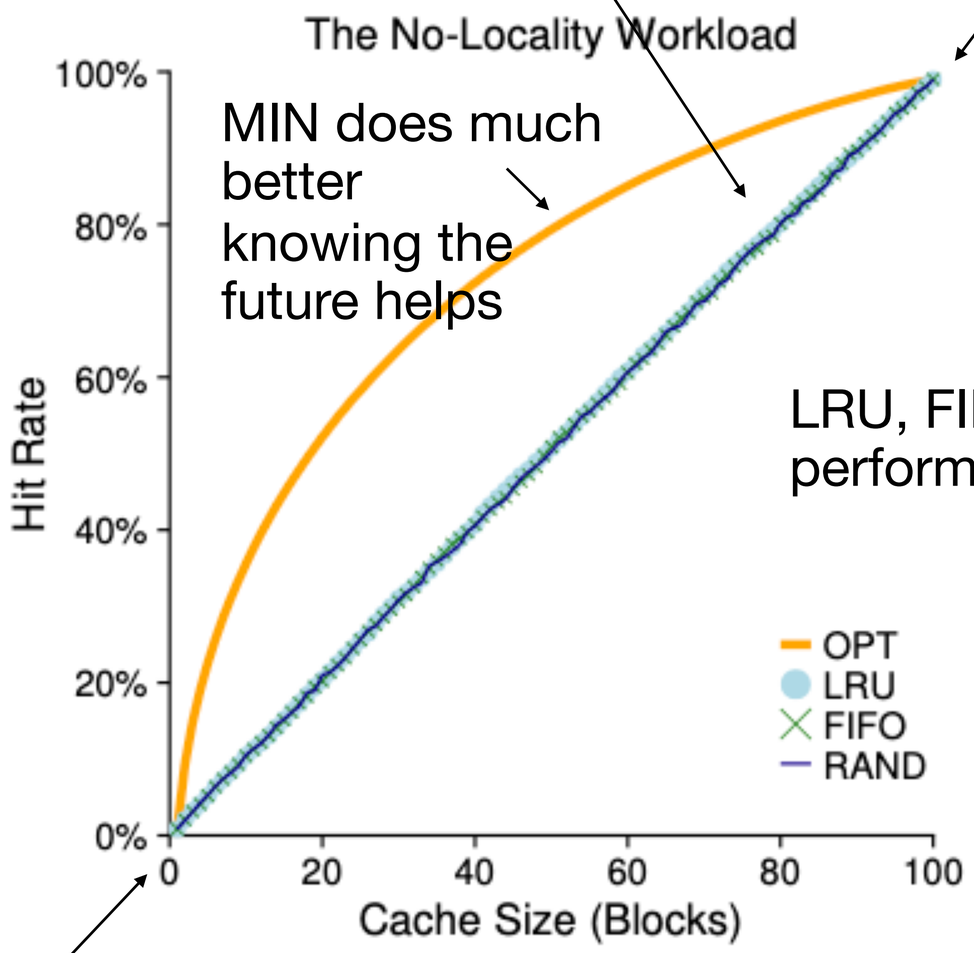
then repeat with other algorithms

- Example 1: **no locality**
  - a program references 100 **unique random** pages
  - 10,000 accesses
- vary the cache size
  - 1 - smallest possible cache
  - 100 - cache that fits all unique pages
- measure hit rate for every run



Benchmark

- Example 1: no locality
  - a program references 100 unique random pages
  - 10,000 accesses
- vary the cache size
  - 1 - smallest possible cache
  - 100 - cache that fits all unique pages
- measure hit rate for every run



performance improves as the cache grows

best performance when all blocks fit into cache

MIN does much better knowing the future helps

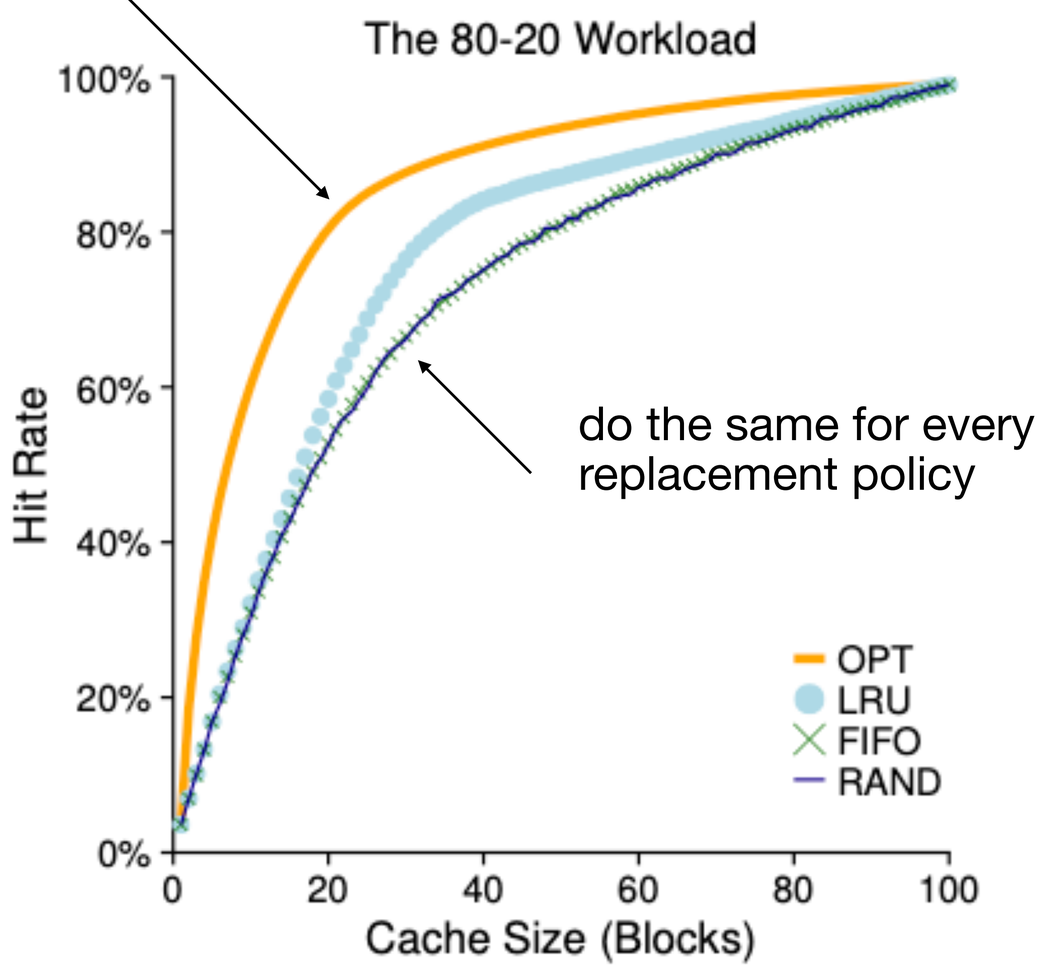
LRU, FIFO, RAND perform the same

when the cache is 1 even knowing the future does not help

Benchmark

run program with 100 different caches sizes and MIN policy -> compute hit rate -> plot a dot for each run

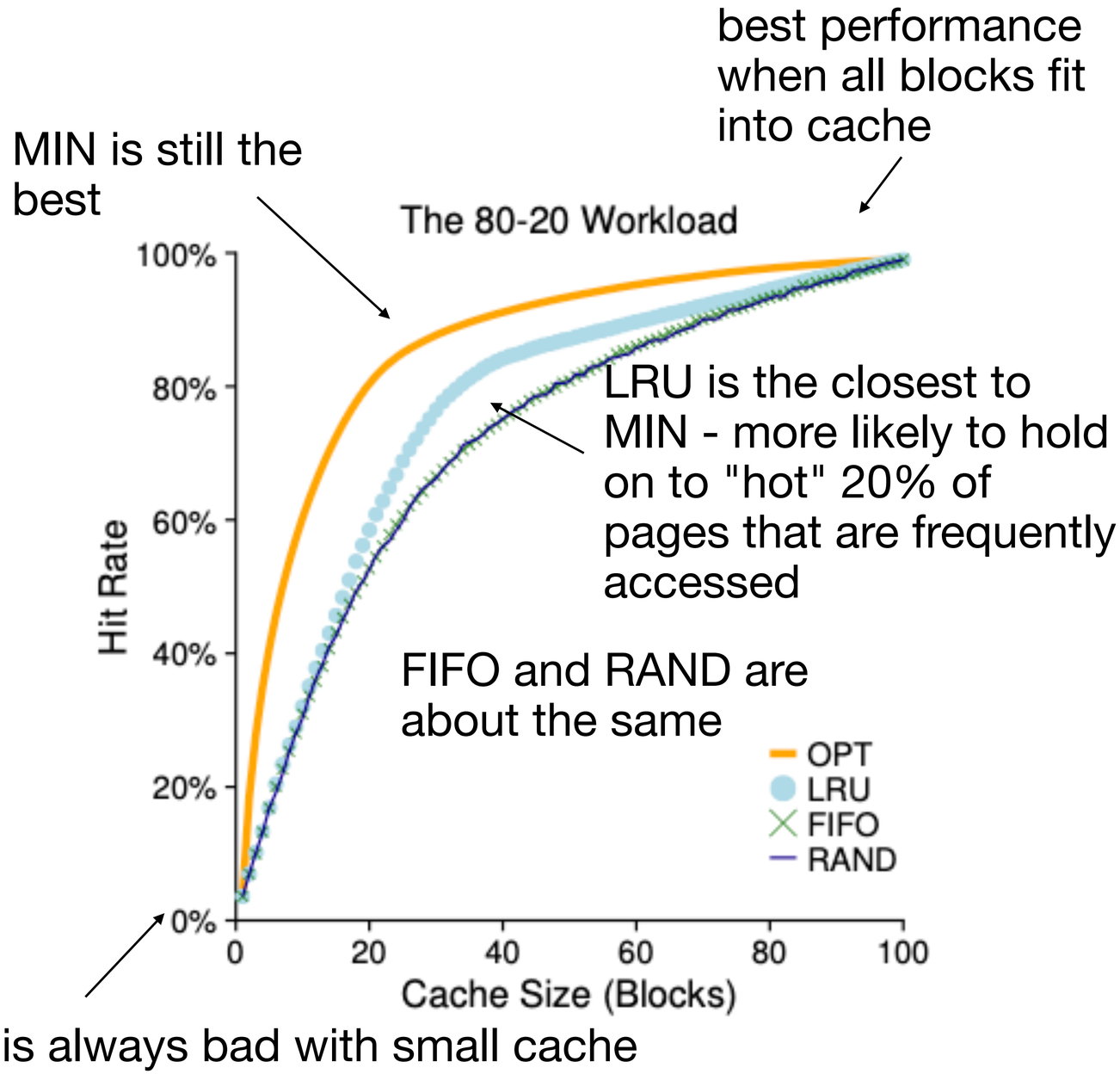
- Example 2: **80-20 workload**
  - a program references 100 unique pages
  - 10,000 accesses: 80% of accesses made to 20% of the pages
  - 20% of pages are "hot" and 80% of pages are "cold"
- vary the cache size
  - 1 - smallest possible cache
  - 100 - cache that fits all unique pages
- measure hit rate after every run



do the same for every replacement policy

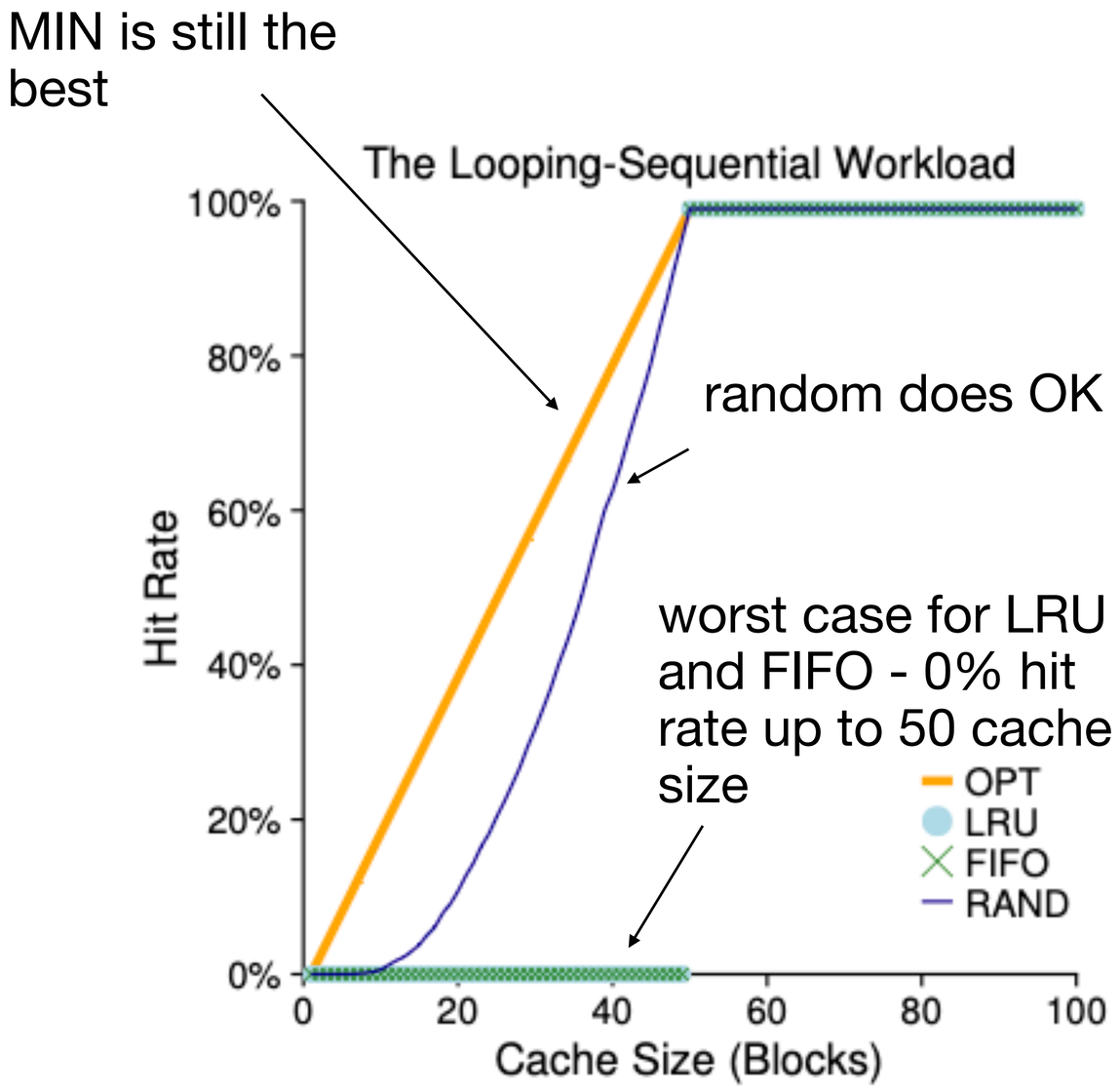
Benchmark

- Example 2: 80-20 workload
  - a program references 100 unique pages
  - 10,000 accesses: 80% of accesses made to 20% of the pages
  - 20% of pages are "hot" and 80% of pages are "cold"
- vary the cache size
  - 1 - smallest possible cache
  - 100 - cache that fits all unique pages
- measure hit rate after every run



Benchmark

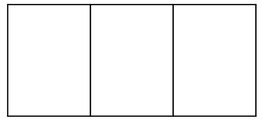
- Example 3: looping sequence
  - a program references 50 pages in sequence (0, 1, 2, ..50) in a loop
  - a typical load in many applications (e.g. databases)
  - 10,000 accesses
- vary the cache size
  - 1 - smallest possible cache
  - 100 - cache that fits all unique pages
- measure hit rate after every run



Benchmark

- Example 3: looping sequence
  - a program references 50 pages in sequence (0, 1, 2, ..50) in a loop
  - a typical load in many applications (e.g. databases)
  - 10,000 accesses
- vary the cache size
  - 1 - smallest possible cache
  - 100 - cache that fits all unique pages
- measure hit rate after every run

FIFO demo



0 1 2 3 4 0

access 4 pages in a loop with 3-page cache

Benchmark

- Example 3: looping sequence
  - a program references 50 pages in sequence (0, 1, 2, ..50) in a loop
  - a typical load in many applications (e.g. databases)
  - 10,000 accesses
- vary the cache size
  - 1 - smallest possible cache
  - 100 - cache that fits all unique pages
- measure hit rate after every run

FIFO demo

0		

1 2 3 4 0 1

f

access 4 pages in a loop with 3-page cache

Benchmark

- Example 3: looping sequence
  - a program references 50 pages in sequence (0, 1, 2, ..50) in a loop
  - a typical load in many applications (e.g. databases)
  - 10,000 accesses
- vary the cache size
  - 1 - smallest possible cache
  - 100 - cache that fits all unique pages
- measure hit rate after every run

FIFO demo

0	1	

2 3 4 0 1 2

f f

access 4 pages in a loop with 3-page cache

Benchmark

- Example 3: looping sequence
  - a program references 50 pages in sequence (0, 1, 2, ..50) in a loop
  - a typical load in many applications (e.g. databases)
  - 10,000 accesses
- vary the cache size
  - 1 - smallest possible cache
  - 100 - cache that fits all unique pages
- measure hit rate after every run

FIFO demo

0	1	2

3 4 0 1 2 3

f f f

access 4 pages in a loop with 3-page cache

Benchmark

- Example 3: looping sequence
  - a program references 50 pages in sequence (0, 1, 2, ..50) in a loop
  - a typical load in many applications (e.g. databases)
  - 10,000 accesses
- vary the cache size
  - 1 - smallest possible cache
  - 100 - cache that fits all unique pages
- measure hit rate after every run

FIFO demo

1	2	3

4 0 1 2 3 4

f f f f

access 4 pages in a loop with 3-page cache

Benchmark

- Example 3: looping sequence
  - a program references 50 pages in sequence (0, 1, 2, ..50) in a loop
  - a typical load in many applications (e.g. databases)
  - 10,000 accesses
- vary the cache size
  - 1 - smallest possible cache
  - 100 - cache that fits all unique pages
- measure hit rate after every run

FIFO demo

2	3	4

0 1 2 3 4 0

f f f f f

access 4 pages in a loop with 3-page cache

Benchmark

- Example 3: looping sequence
  - a program references 50 pages in sequence (0, 1, 2, ..50) in a loop
  - a typical load in many applications (e.g. databases)
  - 10,000 accesses
- vary the cache size
  - 1 - smallest possible cache
  - 100 - cache that fits all unique pages
- measure hit rate after every run

FIFO demo

3	4	0

1 2 3 4 0 1

f f f f f f

access 4 pages in a loop with 3-page cache

Benchmark

- Example 3: looping sequence
  - a program references 50 pages in sequence (0, 1, 2, ..50) in a loop
  - a typical load in many applications (e.g. databases)
  - 10,000 accesses
- vary the cache size
  - 1 - smallest possible cache
  - 100 - cache that fits all unique pages
- measure hit rate after every run

FIFO demo

4	0	1

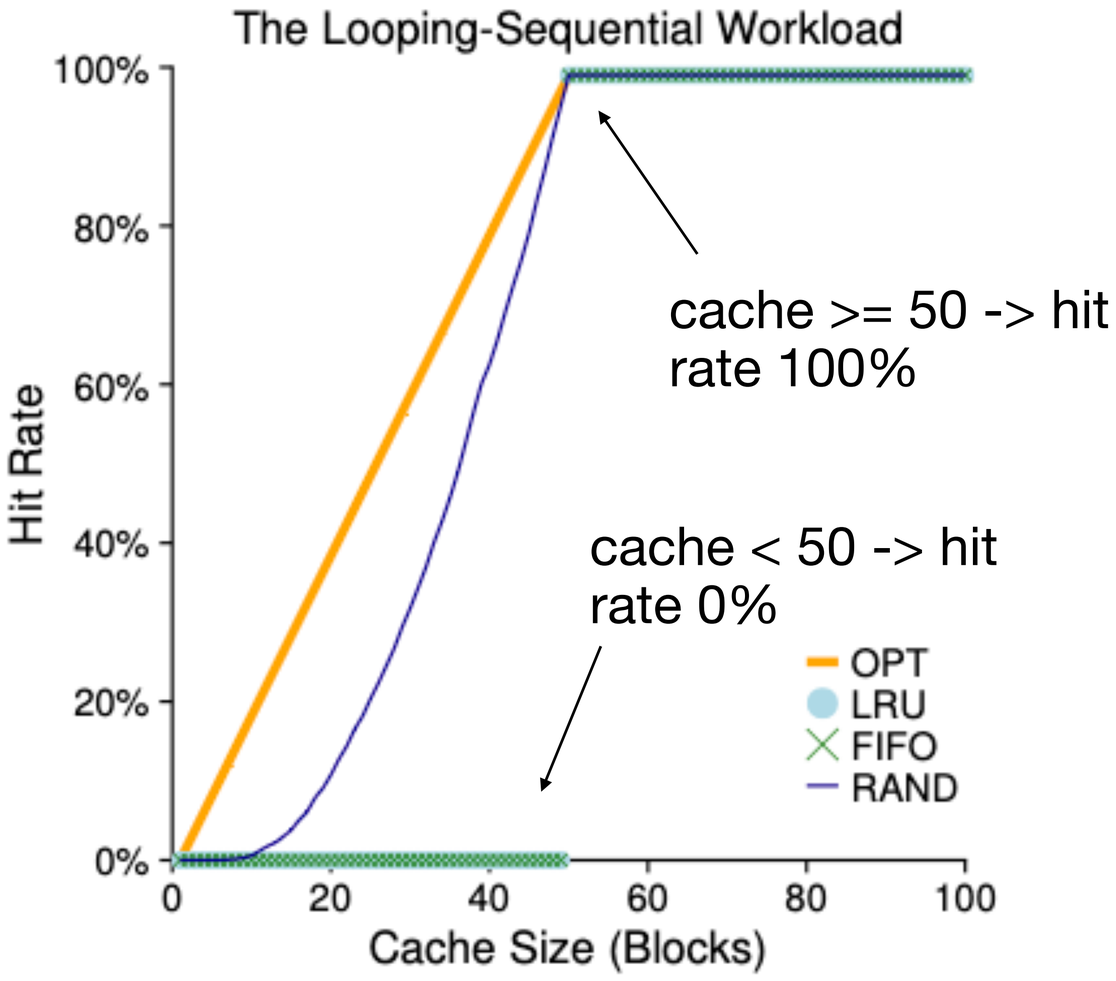
2 3 4 0 1 2

f f f f f f f **0% hit rate forever**

when cache is even **slightly smaller** than pages that the program is looping over

Benchmark

- Example 3: looping sequence
  - a program references 50 pages in sequence (0, 1, 2, ..50) in a loop
  - a typical load in many applications (e.g. databases)
  - 10,000 accesses
- vary the cache size
  - 1 - smallest possible cache
  - 100 - cache that fits all unique pages
- measure hit rate after every run



Summary

- Replacement policies
  - **LRU** - remove **least recently** used pages - use **history** to make eviction decisions
  - **expensive** to implement exactly —> **approximate** instead
    - **clock** algorithm and variations -> use reference and/or dirty bit to decide which page to evict

Summary

- Benchmarking policies
  - use artificial/real workloads + metrics to compare policies
- Some general observations
  - cache too small -> always bad performance
  - cache that fits 100% pages -> always the best
  - MIN always performs better than others -> **knowing the future helps**
  - LRU **relies on temporal locality** -> fails the no locality test
  - LRU and FIFO suffer from the same **edge case**
  - RAND is **not consistent** but also does not suffer from **edge cases**
    - always performs **between** MIN and others -> good as a **lower bound**

Reading

- Chapter 22 Beyond Physical Memory: Policies
  - https://pages.cs.wisc.edu/~remzi/OSTEP/vm-beyondphys-policy.pdf
- Code
  - https://github.com/remzi-arpacidusseau/ostep-homework/tree/master/vm-beyondphys-policy
- No optional reading this week

Quiz grades

- Kahoot quietly changed the format of their reports
- This broke all my scripts that tally the grades and import them to CourseLink
- Will fix by the end of the week and post the grades for last and this week

A1

- Code grades are posted
- README is still being graded

A2

- Due this Sunday, Feb 15 @ 11:59 PM
- If you used grace days for A1, you have less than 4 now
- 4 grace days are for the whole term **not per-assignment**
- Submissions after the deadline + remaining grace days will receive a grade 0

A3

- A3 will be posted on Sunday
- Due on March 8th
  - 3 weeks from now
  - 1 week after the midterm
- Start working on it sooner than later
- Will help for midterm

Midterm

- Feb 26 during class time — 80 minutes
- In-person, closed book
  - 20 multiple choice questions
  - 5 short answer questions
- covers all material taught so far
  - lectures 1-11, labs, assignments
- Feb 24 lecture is midterm review
  - there will be an in-class quiz

Midterm - how to study

- Midterm will focus on **understanding** and **applying** concepts
- Multiple choice questions will be similar to in-class quizzes
- Short answer questions will be similar to exercises during lectures
  - e.g. find problems inx C code, trace scheduling algos, trace cache replacement algos
- Use quizzes to guide your studying
  - Do not memorize the answers - questions will **not** be repeated
  - For each question, try to **explain** why some options are wrong and why one is correct
  - If you **cannot explain** the concept/answer in your own words, you probably do not **understand** it

Quiz 10

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999